Max Gap Between Two Same Characters

Problem Statement

Given a string `s`, find the **maximum number of characters between two occurrences of the same character**.

If no character appears more than once, return `-1`.

Examples

Example 1

```
s = "socks"
```

Indexes:

```
s o c k s
0 1 2 3 4
```

For character `'s'`:

```
gap = 4 - 0 - 1 = 3
```

Answer:

```
3
```

Example 2

```
s = "aa"
```

```
a a
0 1
```

Gap:

```
1 - 0 - 1 = 0
```

Answer:

```
0
```

Example 3

```
s = "abc"
```

No repeated characters.

Answer:

```
-1
```

Understanding the Problem

The keyword is:

```
MAX GAP
```

Not:

```
same characters
```

Gap means:

```
distance
between
farthest
difference
```

Whenever you see these words, think:

```
INDEXES
```

NOT frequency.

Common Mistake

Most people do:

```
from collections import Counter
freq = Counter(s)
```

because they see:

```
same characters
→ frequency
```

But frequency cannot tell us:

```
where the characters occur
```

Example:

```
Counter("socks")
```

Output:

```
{'s':2,'o':1,'c':1,'k':1}
```

Can we compute the gap from this?

No.

Because we need:

```
first index
last index
```

Manual Thinking (Very Important)

Suppose:

```
s = "socks"
```

How would you solve it on paper?

1. Look at index 0 (`s`)
2. Search to the right.
3. Found another `s` at index 4.
4. Gap:

```
4 - 0 - 1 = 3
```

This immediately gives the brute force idea.

Brute Force Approach

Idea

Try every pair of indexes.

If characters are same:

```
gap = j - i - 1
```

Update answer.

Building the Logic

```
for every i
    for every j > i
        if s[i] == s[j]
              gap = j - i - 1
              answer = max(answer, gap)
```

Brute Force Code

```
def maxCharGap(s):
    ans = -1
    n = len(s)

    for i in range(n):
        for j in range(i + 1, n):
            if s[i] == s[j]:
                ans = max(ans, j - i - 1)

    return ans
```

Dry Run

```
s = "socks"
```

i = 0

```
j = 1 → o ≠ s
j = 2 → c ≠ s
j = 3 → k ≠ s
j = 4 → s = s
```

Gap:

```
4 - 0 - 1 = 3
```

Answer:

```
ans = 3
```

No other repeated characters.

Final Answer:

```
3
```

Complexity

Time

```
O(n²)
```

Space

```
O(1)
```

Optimization

Observe:

For every character, we only need:

```
first occurrence
current occurrence
```

We do NOT need all occurrences.

Optimal Idea

Maintain a hashmap:

```
first[ch] = first index of ch
```

While traversing:

First time seeing character

Store its index.

Seen before

Compute:

```
gap = current_index - first_index - 1
```

Update answer.

Dry Run

```
s = "socks"
```

Initial:

```
first = {}
ans = -1
```

---

i = 0

```
ch = 's'
first = {'s':0}
```

---

i = 1

```
ch = 'o'
first = {'s':0,'o':1}
```

---

i = 2

```
ch = 'c'
first = {'s':0,'o':1,'c':2}
```

---

i = 3

```
ch = 'k'
first = {'s':0,'o':1,'c':2,'k':3}
```

---

i = 4

```
ch = 's'
```

Already exists.

Gap:

```
4 - 0 - 1 = 3
```

Update:

```
ans = 3
```

Optimal Code

```
def maxCharGap(s):
    first = {}
    ans = -1

    for i, ch in enumerate(s):
        if ch not in first:
            first[ch] = i
        else:
            gap = i - first[ch] - 1
            ans = max(ans, gap)

    return ans
```

Complexity Analysis

Time

```
O(n)
```

Space

```
O(k)
```

where:

```
k = number of distinct characters
```

Why We Store First Occurrence Only

Suppose:

```
s = "abcaabcd"
```

Indexes:

```
a b c a a b c d
0 1 2 3 4 5 6 7
```

For character `'a'`:

```
0,3,4
```

Maximum gap:

```
4 - 0 - 1 = 3
```

Keeping the earliest occurrence gives the largest distance.

So we never update:

```
first['a']
```

Important Interview Learning

Whenever the problem contains:

```
distance
gap
between
nearest
farthest
length between positions
```

Immediately think:

```
INDEXES
```

NOT:

```
frequency
sorting
counting
```

Pattern Learned

Pattern Name

```
First Occurrence HashMap Pattern
```

Template

```
mp = {}
ans = ...

for i, x in enumerate(arr):
```

```
    if x not in mp:
        mp[x] = i

    else:
        # use previous index
        something = i - mp[x]
        ans = update(ans, something)
```

When to Recognize This Pattern

Questions involving:

- first occurrence
- last occurrence
- maximum distance
- minimum distance
- repeated elements
- nearest duplicate
- farthest duplicate
- longest span

General Template

```
mp = {}

for i in range(n):

    if arr[i] not in mp:
        mp[arr[i]] = i
    else:
        previous = mp[arr[i]]

        # calculate answer

        # update if necessary
```

Similar Problems

---

1. Largest Subarray with Equal 0s and 1s

Leetcode 525

Pattern:

```
Prefix Sum + First Occurrence HashMap
```

---

2. Longest Consecutive Sequence

Leetcode 128

Pattern:

```
HashSet
```

---

3. Contains Duplicate II

Leetcode 219

Pattern:

```
Store last occurrence index.
```

---

4. Longest Substring Without Repeating Characters

Leetcode 3

Pattern:

```
Last occurrence + Sliding Window
```

---

5. First Unique Character in a String

Leetcode 387

Pattern:

```
Frequency + Index
```

---

6. Degree of an Array

Leetcode 697

Pattern:

```
Frequency + First Index + Last Index
```

---

7. Maximum Distance Between Equal Elements

Pattern:

```
Store first occurrence and maximize distance.
```

Final Takeaway

For every problem, ask these three questions:

```
1. What exactly am I returning?
2. What information is needed to compute it?
3. How would I solve it manually on paper?
```

Then follow:

```
Manual Solution
        ↓
Brute Force
        ↓
Optimization
```

This process is how almost every good DSA solution is discovered.